

✓ Brute Force - Geometric & Exhaustive Search Problems

```
from math import sqrt
from itertools import permutations, combinations
```

✓ TOPIC 1: CLOSEST PAIR OF POINTS

```
def euclidean_distance(p1, p2):
    return sqrt((p1[0] - p2[0]) ** 2 +
                (p1[1] - p2[1]) ** 2)

def closest_pair(points):
    min_distance = float('inf')
    closest = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            distance = euclidean_distance(points[i], points[j])

            if distance < min_distance:
                min_distance = distance
                closest = (points[i], points[j])

    return closest, min_distance
```

✓ QUESTION 1

```
print("\n===== CLOSEST PAIR Q1 =====")

points = [(1, 2), (4, 5), (7, 8), (3, 1)]

pair, distance = closest_pair(points)

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

```
===== CLOSEST PAIR Q1 =====
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
```

✓ QUESTION 2

```
print("\n===== CLOSEST PAIR Q2 =====")

points = [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

pair, distance = closest_pair(points)
```

```
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

```
===== CLOSEST PAIR Q2 =====
Closest pair: (5, 4) - (4, 6)
Minimum distance: 2.23606797749979
```

QUESTION 3

```
print("\n===== CLOSEST PAIR Q3 =====")

points = [(0, 0), (2, 2), (3, 1), (6, 5)]

pair, distance = closest_pair(points)

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

```
===== CLOSEST PAIR Q3 =====
Closest pair: (2, 2) - (3, 1)
Minimum distance: 1.4142135623730951
```

QUESTION 4

```
print("\n===== CLOSEST PAIR Q4 =====")

points = [(10, 20), (15, 25), (30, 40), (12, 22)]

pair, distance = closest_pair(points)

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

```
===== CLOSEST PAIR Q4 =====
Closest pair: (10, 20) - (12, 22)
Minimum distance: 2.8284271247461903
```

QUESTION 5: CLOSEST PAIR + COMPLEXITY

```
print("\n===== CLOSEST PAIR Q5 =====")

points = [
    (10, 0),
    (11, 5),
    (5, 3),
    (9, 3.5),
    (15, 3),
    (12.5, 7),
    (6, 6.5),
    (7.5, 4.5)
]
```

```

pair, distance = closest_pair(points)

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)

print("\nTime Complexity:")
print("Brute Force Closest Pair =  $O(n^2)$ ")

print("Space Complexity:")
print("O(1) auxiliary space")

```

```

===== CLOSEST PAIR Q5 =====
Closest pair: (9, 3.5) - (7.5, 4.5)
Minimum distance: 1.8027756377319946

Time Complexity:
Brute Force Closest Pair =  $O(n^2)$ 
Space Complexity:
O(1) auxiliary space

```

✓ CONVEX HULL BASIC BRUTE FORCE (QUESTION 5 CONTINUED)

```

def orientation(p, q, r):
    value = ((q[0] - p[0]) * (r[1] - p[1]) -
             (q[1] - p[1]) * (r[0] - p[0]))

    if value > 0:
        return 1      # Counter-clockwise

    if value < 0:
        return -1     # Clockwise

    return 0          # Collinear

def convex_hull_brute(points):
    n = len(points)

    if n <= 2:
        return points

    hull_edges = set()

    for i in range(n):
        for j in range(i + 1, n):

            left = False
            right = False

            for k in range(n):

                if k == i or k == j:
                    continue

                side = orientation(points[i], points[j], points[k])

                if side > 0:

```

```

        left = True

    elif side < 0:
        right = True

    # Edge belongs to hull if all other points
    # are on one side or collinear.
    if not (left and right):
        hull_edges.add((i, j))

# Collect hull points
hull_points = set()

for i, j in hull_edges:
    hull_points.add(points[i])
    hull_points.add(points[j])

# Arrange hull points around centroid
center_x = sum(p[0] for p in hull_points) / len(hull_points)
center_y = sum(p[1] for p in hull_points) / len(hull_points)

def angle(p):
    import math
    return math.atan2(p[1] - center_y, p[0] - center_x)

return sorted(hull_points, key=angle)

print("\n===== CONVEX HULL Q5 =====")

points = [
    (10, 0),      # P1
    (11, 5),      # P2
    (5, 3),       # P3
    (9, 3.5),     # P4
    (15, 3),      # P5
    (12.5, 7),    # P6
    (6, 6.5),     # P7
    (7.5, 4.5)    # P8
]

hull = convex_hull_brute(points)

print("Convex Hull Points:")

for p in hull:
    print(p)

print("\nTime Complexity: O(n^3)")
print("Space Complexity: O(n)")

```

```

===== CONVEX HULL Q5 =====
Convex Hull Points:
(5, 3)
(10, 0)
(15, 3)
(12.5, 7)
(6, 6.5)

```

Time Complexity: $O(n^3)$
 Space Complexity: $O(n)$

✓ HANDLING COLLINEAR POINTS

```
def convex_hull_with_collinear(points):
    """
    Brute-force convex hull that removes interior collinear
    points and keeps only extreme boundary points.
    """

    n = len(points)

    if n <= 2:
        return points

    hull = set()

    for i in range(n):
        for j in range(i + 1, n):

            left = False
            right = False

            for k in range(n):
                if k == i or k == j:
                    continue

                side = orientation(points[i], points[j], points[k])

                if side > 0:
                    left = True
                elif side < 0:
                    right = True

            if not (left and right):
                hull.add(points[i])
                hull.add(points[j])

    center_x = sum(p[0] for p in hull) / len(hull)
    center_y = sum(p[1] for p in hull) / len(hull)

    import math

    hull = sorted(
        hull,
        key=lambda p: math.atan2(
            p[1] - center_y,
            p[0] - center_x
        )
    )

    return hull
```

✓ TOPIC 2: CONVEX HULL

✓ QUESTION 6

```
print("\n===== CONVEX HULL Q6 =====")

points = [
    (0, 0),
    (1, 1),
    (2, 2),
    (0, 2),
    (2, 0)
]

hull = convex_hull_with_collinear(points)

print("Convex Hull Points:")
for p in hull:
    print(p)
```

```
===== CONVEX HULL Q6 =====
Convex Hull Points:
(0, 0)
(2, 0)
(2, 2)
(0, 2)
```

✓ QUESTION 7

```
print("\n===== CONVEX HULL Q7 =====")

points = [
    (1, 2),
    (3, 1),
    (5, 3),
    (4, 6),
    (2, 5)
]

hull = convex_hull_with_collinear(points)

print("Convex Hull Points:")
for p in hull:
    print(p)
```

```
===== CONVEX HULL Q7 =====
Convex Hull Points:
(1, 2)
(3, 1)
(5, 3)
(4, 6)
(2, 5)
```

✓ QUESTION 8

```
print("\n===== CONVEX HULL Q8 =====")

points = [
    (0, 3),
    (1, 1),
    (2, 2),
    (4, 4),
    (3, 0)
]

hull = convex_hull_with_collinear(points)

print("Convex Hull Points:")
for p in hull:
    print(p)
```

```
===== CONVEX HULL Q8 =====
Convex Hull Points:
(1, 1)
(3, 0)
(4, 4)
(0, 3)
```

QUESTION 9

```
print("\n===== CONVEX HULL Q9 =====")

points = [
    (2, 2),
    (4, 1),
    (6, 3),
    (5, 5),
    (3, 6),
    (1, 4)
]

hull = convex_hull_with_collinear(points)

print("Convex Hull Points:")
for p in hull:
    print(p)
```

```
===== CONVEX HULL Q9 =====
Convex Hull Points:
(2, 2)
(4, 1)
(6, 3)
(5, 5)
(3, 6)
(1, 4)
```

QUESTION 10

```
print("\n===== CONVEX HULL Q10 =====")
```

```

points = [
    (1, 1),
    (2, 4),
    (5, 5),
    (6, 2),
    (3, 0)
]

hull = convex_hull_with_collinear(points)

print("Convex Hull Points:")
for p in hull:
    print(p)

```

```

===== CONVEX HULL Q10 =====
Convex Hull Points:
(1, 1)
(3, 0)
(6, 2)
(5, 5)
(2, 4)

```

✓ TOPIC 3: EXHAUSTIVE SEARCH FOR SUBSET SUM

```

def subset_sum_exhaustive(numbers, target):
    n = len(numbers)

    for r in range(n + 1):

        for subset in combinations(numbers, r):

            if sum(subset) == target:
                return list(subset)

    return None

```

✓ QUESTION 11

```

print("\n===== SUBSET SUM Q11 =====")

numbers = [3, 34, 4, 12, 5, 2]
target = 9

subset = subset_sum_exhaustive(numbers, target)

if subset:
    print("Subset found:", subset)
    print("Sum:", sum(subset))
else:
    print("No subset found")

```

```

===== SUBSET SUM Q11 =====
Subset found: [4, 5]
Sum: 9

```

QUESTION 12: EXHAUSTIVE SEARCH - TSP

```
def calculate_path_distance(path):
    distance = 0

    for i in range(len(path) - 1):
        distance += euclidean_distance(path[i], path[i + 1])

    return distance

def tsp_exhaustive(cities):
    start = cities[0]

    best_distance = float('inf')
    best_path = None

    remaining = cities[1:]

    for perm in permutations(remaining):

        path = [start] + list(perm) + [start]

        distance = calculate_path_distance(path)

        if distance < best_distance:
            best_distance = distance
            best_path = path

    return best_path, best_distance

print("\n===== TSP Q12 - TEST CASE 1 =====")

cities1 = [
    (1, 2),
    (4, 5),
    (7, 1),
    (3, 6)
]

path, distance = tsp_exhaustive(cities1)

print("Shortest Distance:", distance)
print("Shortest Path:", path)

print("\n===== TSP Q12 - TEST CASE 2 =====")

cities2 = [
    (2, 4),
    (8, 1),
    (1, 7),
    (6, 3),
    (5, 9)
]
```

```
path, distance = tsp_exhaustive(cities2)
```

```
print("Shortest Distance:", distance)
print("Shortest Path:", path)
```

```
===== TSP Q12 - TEST CASE 1 =====
```

```
Shortest Distance: 16.969112047670894
```

```
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
```

```
===== TSP Q12 - TEST CASE 2 =====
```

```
Shortest Distance: 23.12995011084934
```

```
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]
```

✓ QUESTION 13: EXHAUSTIVE SEARCH - ASSIGNMENT PROBLEM

```
def total_cost(assignment, cost_matrix):
    total = 0

    for worker, task in assignment:
        total += cost_matrix[worker][task]

    return total
```

```
def assignment_problem(cost_matrix):
    n = len(cost_matrix)

    best_cost = float('inf')
    best_assignment = None

    for task_permutation in permutations(range(n)):

        assignment = []

        for worker in range(n):
            task = task_permutation[worker]
            assignment.append((worker, task))

        cost = total_cost(assignment, cost_matrix)

        if cost < best_cost:
            best_cost = cost
            best_assignment = assignment

    return best_assignment, best_cost
```

```
print("\n===== ASSIGNMENT PROBLEM Q13 - TEST CASE 1 =====")
```

```
cost_matrix1 = [
    [3, 10, 7],
    [8, 5, 12],
    [4, 6, 9]
]
```

```
assignment, cost = assignment_problem(cost_matrix1)
```

```

print("Optimal Assignment:")

for worker, task in assignment:
    print(
        f"(worker {worker + 1}, task {task + 1})"
    )

print("Total Cost:", cost)

print("\n===== ASSIGNMENT PROBLEM Q13 - TEST CASE 2 =====")

cost_matrix2 = [
    [15, 9, 4],
    [8, 7, 18],
    [6, 12, 11]
]

assignment, cost = assignment_problem(cost_matrix2)

print("Optimal Assignment:")

for worker, task in assignment:
    print(
        f"(worker {worker + 1}, task {task + 1})"
    )

print("Total Cost:", cost)

```

```

===== ASSIGNMENT PROBLEM Q13 - TEST CASE 1 =====
Optimal Assignment:
(worker 1, task 3)
(worker 2, task 2)
(worker 3, task 1)
Total Cost: 16

===== ASSIGNMENT PROBLEM Q13 - TEST CASE 2 =====
Optimal Assignment:
(worker 1, task 3)
(worker 2, task 2)
(worker 3, task 1)
Total Cost: 17

```

✓ QUESTION 14: EXHAUSTIVE SEARCH - 0/1 KNAPSACK

```

def total_value(items, values):
    total = 0

    for item in items:
        total += values[item]

    return total

def is_feasible(items, weights, capacity):
    total_weight = 0

    for item in items:

```

```
        total_weight += weights[item]

    return total_weight <= capacity

def knapsack_exhaustive(weights, values, capacity):
    n = len(weights)

    best_items = []
    best_value = 0

    for r in range(n + 1):

        for subset in combinations(range(n), r):

            if is_feasible(subset, weights, capacity):

                value = total_value(subset, values)

                if value > best_value:
                    best_value = value
                    best_items = list(subset)

    return best_items, best_value

print("\n===== KNAPSACK Q14 - TEST CASE 1 =====")

weights1 = [2, 3, 1]
values1 = [4, 5, 3]
capacity1 = 4

items, value = knapsack_exhaustive(
    weights1,
    values1,
    capacity1
)

print("Optimal Selection:", items)
print("Total Value:", value)

print("\n===== KNAPSACK Q14 - TEST CASE 2 =====")

weights2 = [1, 2, 3, 4]
values2 = [2, 4, 6, 3]
capacity2 = 6

items, value = knapsack_exhaustive(
    weights2,
    values2,
    capacity2
)

print("Optimal Selection:", items)
print("Total Value:", value)
```

```
===== KNAPSACK Q14 - TEST CASE 1 =====
```

```
Optimal Selection: [1, 2]
Total Value: 8
```

```
===== KNAPSACK Q14 - TEST CASE 2 =====
Optimal Selection: [0, 1, 2]
Total Value: 12
```

✓ QUESTION 15: SUBSET SUM - ALL POSSIBLE SUBSETS

```
def all_subset_sum(numbers, target):
    n = len(numbers)
    found = []

    for r in range(n + 1):

        for subset in combinations(numbers, r):

            if sum(subset) == target:
                found.append(list(subset))

    return found

print("\n===== SUBSET SUM Q15 =====")

numbers = [15, 10, 12, 7, 5]
target = 22

subsets = all_subset_sum(numbers, target)

if subsets:

    for subset in subsets:
        print("Subset found:", subset)
        print("Sum:", sum(subset))

else:
    print("No subset found")
```

```
===== SUBSET SUM Q15 =====
Subset found: [15, 7]
Sum: 22
Subset found: [10, 12]
Sum: 22
Subset found: [10, 7, 5]
Sum: 22
```

✓ COMPLEXITY ANALYSIS

◆ Gemini

```
print("\n=====")
print("COMPLEXITY ANALYSIS")
print("=====")

print("\n1. Closest Pair - Brute Force")
```

```

print("Time Complexity :  $O(n^2)$ ")
print("Space Complexity:  $O(1)$  auxiliary space")

print("\n2. Convex Hull - Basic Brute Force")
print("Time Complexity :  $O(n^3)$ ")
print("Space Complexity:  $O(n)$ ")

print("\n3. TSP - Exhaustive Search")
print("Time Complexity :  $O(n!)$ ")
print("Space Complexity:  $O(n)$ ")

print("\n4. Assignment Problem - Exhaustive Search")
print("Time Complexity :  $O(n!)$ ")
print("Space Complexity:  $O(n)$ ")

print("\n5. 0/1 Knapsack - Exhaustive Search")
print("Time Complexity :  $O(2^n)$ ")
print("Space Complexity:  $O(n)$ ")

print("\n6. Subset Sum - Exhaustive Search")
print("Time Complexity :  $O(2^n)$ ")
print("Space Complexity:  $O(n)$ ")

```

```

=====
COMPLEXITY ANALYSIS
=====

```

```

1. Closest Pair - Brute Force
Time Complexity :  $O(n^2)$ 
Space Complexity:  $O(1)$  auxiliary space

2. Convex Hull - Basic Brute Force
Time Complexity :  $O(n^3)$ 
Space Complexity:  $O(n)$ 

3. TSP - Exhaustive Search
Time Complexity :  $O(n!)$ 
Space Complexity:  $O(n)$ 

4. Assignment Problem - Exhaustive Search
Time Complexity :  $O(n!)$ 
Space Complexity:  $O(n)$ 

5. 0/1 Knapsack - Exhaustive Search
Time Complexity :  $O(2^n)$ 
Space Complexity:  $O(n)$ 

6. Subset Sum - Exhaustive Search
Time Complexity :  $O(2^n)$ 
Space Complexity:  $O(n)$ 

```

